

A
PROJECT REPORT ON

Exploring Learning Analytics of Skill-Based Coursing using Machine Learning Classification Models

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

By
N GANESH RAHUL
(24BFF00075)

Under the esteemed guidance of

J. Kiran Kumar, MCA (Ph.D)
Associate Professor



DEPARTMENT OF COMPUTER APPLICATIONS

**SRI VENKATESWARA COLLEGE OF ENGINEERING
(AUTONOMOUS)**

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Anantapur)

Accredited by NAAC with 'A' Grade

Opp. LIC Training Centre, Karakambadi Road, TIRUPATI-517507

2024-2026

**SRI VENKATESWARA COLLEGE OF ENGINEERING
(AUTONOMOUS)**

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Anantapur)

Accredited by NAAC with 'A' Grade

Opp. LIC Training Centre, Karakambadi Road, TIRUPATI – 517507

DEPARTMENT OF COMPUTER APPLICATIONS

2021-2023



CERTIFICATE

*This is to certify that the project report entitled “**Exploring Learning Analytics of Skill-Based Coursing using Machine Learning Classification Models**” is a bonafide record of the project work done and submitted by*

N GANESH RAHUL

(24BFF00075)

*for the partial fulfillment of the requirements for the award of **MASTER OF COMPUTER APPLICATIONS** Degree from **SRI VENKATESWARA COLLEGE OF ENGINEERING (AUTONOMOUS)** Affiliated to JNT University Anantapur.*

GUIDE

HEAD OF THE DEPARTMENT

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I am thankful to my guide **J. Kiran Kumar**, Associate. Professor for his valuable guidance and encouragement. His helping attitude and suggestions have helped me in the successful completion of the project.

I would like to express my sincere thanks to **Dr. E. Sreedevi**, Professor, Head of the Department of MCA, for her kind help and encouragement during the course of my study and in the successful completion of the project work.

I express my sincere thanks to **Dr. Vijaya Gunturu**, Principal, S.V College of Engineering, Tirupati.

Successful completion of any project cannot be done without proper support and encouragement. My sincere thanks to the Management for providing all the necessary facilities during the Course of my study.

I would like to thank my parents and friends, who have the greatest contributions in all my achievements, for the great care and blessings in making me successful in all my endeavors.

I would like to express my deep gratitude to all those who helped me directly or indirectly to transform an idea into my working project.

**N GANESH RAHUL
(24BFF00075)**

DECLARATION

I hereby declare that project entitled **“Exploring Learning Analytics of Skill-Based Coursing using Machine Learning Classification Models”** submitted to the Department of COMPUTER APPLICATIONS, **SRI VENKATESWARA COLLEGE OF ENGINEERING, TIRUPATI** in partial fulfillment of Requirement for the award of the degree of **MASTER OF COMPUTER APPLICATIONS**.

This project is the result of my own effort and it has not been submitted to any other University or Institution for the award of any degree other than specified above.

SIGNATURE OF THE STUDENT

ABSTRACT

This study explores learning analytics of a skill-based course using various machine learning classification models, including Random Forest, Logistic Regression, CatBoost, Support Vector Classification (SVC), and Naïve Bayes. The objective is to categorize student outcomes into four classes: Pass, Distinction, Withdrawn, and Fail. These models are trained and validated using historical data from students who previously enrolled in the course, encompassing their engagement levels, assessment scores, and other relevant features that influence learning outcomes. Comparative analysis reveals that each model exhibits unique strengths in predicting different outcome classes, providing valuable insights into student performance and the efficacy of the instructional design. The findings from this investigation offer educators and academic institutions a robust framework for early identification of students at risk of underperformance or withdrawal, thereby enabling timely intervention to enhance student success in skill-based courses. Furthermore, the research contributes to the growing body of knowledge in learning analytics and machine learning applications in education, showcasing the potential of these technologies in fostering a supportive and adaptive learning environment for diverse student populations.

Keywords: Random Forest, Logistic Regression, CatBoost, Support Vector Classification (SVC), and Naïve Bayes.

TABLE OF CONTENTS

S.NO	CONTENTS	PAGE NO
1	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 OBJECTIVE OF THE PROJECT	1
	1.3 DESCRIPTION OF THE PROJECT	3
2	LITERATURE SURVEY	4
3	SYSTEM ANALYSIS	7
	3.1 EXISTING SYSTEM	7
	3.2 PROPOSED SYSTEM	8
	3.3 HARDWARE & SOFTWARE REQUIREMENTS	9
4	SYSTEM DESIGN	10
	4.1 SYSTEM ARCHITECTURE	10
	4.2 UML DIAGRAMS	12
5	IMPLEMENTATION	17
	5.1 DESCRIPTION OF MODULES	17
	5.2 SOFTWARE ENVIRONMENT	20
6	ALGORITHMS	32
7	SYSTEM TESTING	38
	7.1 TYPES OF TESTS	38
	7.2 SYSTEM TEST	39
8	RESULT	41
9	CONCLUSION AND FUTURE WORK	42

S.NO	CONTENTS	PAGENO
10	APPENDIX	43
	10.1 APPENDIX	43
	10.2 SAMPLECODE	50
	10.3 SCREEN SHOTS	55
11	REFERENCES	58

LISTOF FIGURES

FIG NO	NAME OF FIGURES	PAGENO
1	USE CASE DIAGRAM	13
2	CLASS DIAGRAM	14
3	SEQUENCE DIAGRAM	15
4	ACTIVITY DIAGRAM	16

1. INTRODUCTION

1.1 GENERAL

Learning analytics has become an essential aspect of modern education, especially in skill-based courses where student performance depends on practical engagement, participation, and continuous assessment. It involves the collection, analysis, and interpretation of data related to learners and their learning environments. Accurate prediction of student outcomes is important for improving teaching strategies, enhancing learning experiences, and supporting timely academic interventions. However, traditional evaluation methods are often time-consuming, subjective, and may not effectively capture complex learning patterns.

In recent years, machine learning techniques have shown significant potential in analyzing educational data and predicting student performance. In this project, we propose a system for exploring the learning analytics of a skill-based course using machine learning classification models. The system focuses on analyzing historical student data such as engagement levels, assessment scores, and learning behaviors to improve prediction accuracy.

The proposed system utilizes machine learning algorithms such as Logistic Regression, Random Forest, Support Vector Classification (SVC), Naïve Bayes, and CatBoost, which are known for their effectiveness in classification tasks. By applying these models, the system aims to accurately classify student outcomes into categories such as Pass, Distinction, Withdrawn, and Fail, thereby providing meaningful insights into student performance.

Overall, this project aims to enhance the effectiveness of learning analytics in skill-based education and to demonstrate the capability of machine learning techniques in predicting student outcomes, ultimately supporting better academic decision-making and improved learning experiences.

1.2 OBJECTIVE OF THE PROJECT:

The objective of using machine learning in learning analytics is to develop a system that can accurately predict student outcomes in skill-based courses based on historical data. Predicting student performance helps in understanding learning patterns and supports effective academic decision-making.

The key objectives of this project would include:

Accurate Prediction:

Machine learning models can analyze student data and predict outcomes such as Pass, Distinction, Withdrawn, and Fail with improved accuracy. Accurate predictions help educators take timely actions and improve student performance.

Learning Analysis:

The system should analyze various factors such as student engagement, assessment scores, and learning behavior to understand their impact on academic performance. This helps in identifying key factors influencing student success.

Decision Support:

The system can assist educators, academic institutions, and administrators by providing reliable predictions, enabling better decisions related to teaching strategies, student support, and course improvement.

Scalability:

The system should be capable of handling large volumes of student data and should adapt to new data inputs over time, ensuring consistent performance and usability.

It is essential to ensure proper data collection, preprocessing, and validation techniques to maintain the reliability and accuracy of the machine learning models.

1.3 PROJECT INTRODUCTION:

Working Methodology

Learning analytics using machine learning involves the development of predictive models that can estimate student outcomes based on historical data. The process includes several important steps that ensure accurate prediction and analysis of student performance:

Data Collection:

The first step is to collect relevant student-related data, which may include different types of information such as:

Student Performance Data: Historical records of student results and final outcomes.

Engagement Data: Information about attendance, participation, and activity levels.

Assessment Data: Scores obtained in assignments, quizzes, and examinations.

Data Preprocessing:

After collecting the data, it must be cleaned and prepared for analysis. This step includes

handling missing values, normalizing or scaling numerical data, encoding categorical variables, and removing inconsistencies or noise that may affect the model's performance.

Feature Engineering:

In this stage, important features are selected or created from the dataset that contribute significantly to predicting student outcomes. Proper feature selection improves the efficiency and accuracy of machine learning models.

Model Selection:

Various machine learning algorithms can be applied for classification tasks, including:

Logistic Regression: Used for predicting categorical outcomes.

Random Forest: Effective in handling complex data and improving prediction accuracy.

Support Vector Classification (SVC): Useful for classification with clear decision boundaries.

Naïve Bayes: Efficient for probabilistic classification.

Training the Model:

The dataset is divided into training and testing sets. The selected models are trained using the training data to learn the relationship between input features and student outcomes.

Model Evaluation:

The trained models are evaluated using the testing dataset. Performance metrics such as accuracy, precision, recall, and F1-score are used to measure how well the model predicts student outcomes.

Hyperparameter Tuning:

Model parameters are adjusted and optimized to improve performance and achieve better prediction results.

Validation and Deployment:

Once the model achieves satisfactory performance, it can be deployed in a real-world system where users can input data and receive predictions regarding student performance.

It is important to note that the accuracy and reliability of the system depend heavily on the quality of the dataset. Proper data collection, preprocessing, and validation are essential to ensure meaningful and consistent predictions.

2.LITERATURE SURVEY

2.1 Related Work:

[1] Heuer, H., & Breiter, A. (2018). Student success prediction and the trade off between big data and data minimization. DeLFI 2018-Die 16. E Learning Fachtagung Informatik

Predicting student success is crucial for educational institutions to improve learning outcomes and retention rates. While big data provides a wealth of information, allowing for detailed analysis and accurate predictions, it raises concerns regarding privacy, security, and ethical use of student information. On the other hand, the principle of data minimization advocates for collecting only the necessary data, reducing privacy risks but possibly compromising the accuracy and depth of predictions. Balancing the trade-off requires a thoughtful approach. Institutions should adopt privacy-preserving technologies and policies while carefully selecting and analyzing data that is critical for predicting student success, without unnecessarily infringing on privacy. This delicate balance ensures that educators can harness the power of data to support students effectively while upholding ethical standards and legal compliance in data handling and protection

[2] Maulud, D., & Abdulazeez, A. M. (2020). A review on linear regression comprehensive in machine learning. Journal of Applied Science and Technology Trends, 1(4), 140-147.

Linear regression is a fundamental algorithm in machine learning, providing a straightforward approach to predicting a response variable based on one or multiple predictor variables. It establishes a linear relationship between the dependent (target) and independent (feature) variables, often used for forecasting, trend analysis, and determining the significance of predictors. The algorithm works by fitting the best linear relationship (line of best fit or regression line) that minimizes the sum of squared differences (residuals) between observed and predicted values. Simple linear regression handles one predictor, while multiple linear regression deals with two or more. Linear regression is advantageous for its simplicity, ease of interpretation, and computational efficiency. However, its reliance on the assumption of linearity limits its applicability to complex, non-linear relationships in data. Despite this, it

remains a foundational technique, often serving as a starting point for more advanced regression methods in machine learning and statistics.

[3] Peach, R. L., Yaliraki, S. N., Lefevre, D., & Barahona, M. (2019). Data driven unsupervised clustering of online learner behaviour. npj Science of Learning, 4(1), 14.

Employing data-driven methods, this research utilizes unsupervised clustering to categorize online learner behavior effectively. The study leverages extensive datasets encompassing various behavioral metrics of online learners. These metrics include engagement levels, time spent on tasks, participation in discussions, and submission patterns. Without prior labeling, unsupervised clustering algorithms, like K-Means and Hierarchical clustering, are applied to discern patterns and segregate learners into distinct behavior groups. This granular classification provides invaluable insights into learner dynamics, revealing distinct profiles like active participants, passive observers, and irregular contributors. Understanding these profiles is crucial for educators and platform developers to tailor educational content and platform features, fostering an environment conducive to the unique needs and expectations of each learner group. This approach promises a personalized and efficient learning experience, responding dynamically to different behavior patterns observed among online learners.

[4] Waheed, H., Hassan, S. U., Aljohani, N. R., Hardman, J., Alelyani, S., & Nawaz, R. (2020). Predicting academic performance of students from VLE big data using deep learning models. Computers in Human behavior, 104, 106189.

Predicting students' academic performance with precision is crucial for timely intervention and support. With the advent of Virtual Learning Environments (VLEs), there's an influx of data regarding student interactions and engagement patterns. This research leverages deep learning models to analyze this extensive data, offering accurate predictions of students' academic outcomes. Deep learning's ability to discern intricate patterns in large datasets makes it apt for handling the complex, multifaceted data generated by VLEs. The research employs various deep learning architectures, meticulously trained and validated with VLE-generated data, to predict outcomes like pass, fail, withdrawal, or distinction. These predictions, grounded in

students' online behaviors and engagement levels, provide educators with invaluable insights. Consequently, educators can proactively identify and support at-risk students, fostering an environment where all students have the opportunity to succeed academically. This study thereby not only contributes to the academic success of individual students but also advances the field of educational data mining and learning analytics.

[5] Wasif, M., Waheed, H., Aljohani, N. R., & Hassan, S. U. (2019). Understanding student learning behavior and predicting their performance. In Cognitive Computing in Technology-Enhanced Learning (pp. 1-28). IGI Global.

Cognitive Computing in Technology-Enhanced Learning (TEL) plays a pivotal role in understanding and predicting student behavior and performance. By mimicking human reasoning, this advanced technology analyzes students' learning patterns and engagement, providing a nuanced understanding of their academic journey. Cognitive computing integrates various AI technologies, such as machine learning, natural language processing, and data mining, offering personalized learning experiences and real-time feedback. It aids in identifying at-risk students early, allowing for timely intervention to support their learning needs. Furthermore, cognitive computing fosters a more interactive and engaging learning environment, adapting to individual students' cognitive styles and pacing. Through the analysis of historical and real-time data, it generates insights into students' strengths and areas for improvement, supporting educators in devising effective teaching strategies and enhancing the overall educational experience. This approach not only supports student success but also contributes significantly to the innovation and effectiveness of TEL environments.

3. SYSTEM ANALYSIS

EXISTING SYSTEM:

The existing system utilizes various machine learning models, including CatBoost, for learning analytics in a skill-based course. CatBoost, a gradient boosting algorithm, is employed for its efficacy in handling categorical features seamlessly. It is trained and validated using historical data of students, incorporating their engagement, assessment scores, and other relevant features. The system categorizes student outcomes into Pass, Distinction, Withdrawn, and Fail. Through comparative analysis, CatBoost demonstrates unique strengths in predicting different outcome classes, contributing to the early identification of students at risk and facilitating timely intervention to support student success in the course.

DISADVANTAGES OF EXISTING SYSTEM:

- ❖ CatBoost requires extensive computational resources, which might be inefficient for institutions with limited processing power and memory capacities.
- ❖ Its complexity necessitates users to have a deep understanding of the algorithm and parameter tuning, posing a steep learning curve.
- ❖ While CatBoost handles categorical features well, the quality of predictions may significantly decline with sparse or imbalanced data sets.
- ❖ The model's training time can be lengthy, especially with large datasets, potentially slowing down the analytics process and delaying interventions.
- ❖ Limited interpretability of CatBoost models hinders educators' and administrators' comprehension of the predictive factors and decision-making process in student outcome predictions.

PROPOSED SYSTEM:

- ❖ This study employs a comparative multi-model machine learning framework — integrating Random Forest, Logistic Regression, Support Vector Classification (SVC), and Naïve Bayes — to examine behavioral and academic patterns embedded within learning analytics data. Rather than relying on singular algorithmic predictions, the ensemble-style comparative approach enables a robust evaluation of model suitability across varying student demographic.

- ❖ Leveraging longitudinal student records that capture dynamic engagement indicators and progressive assessment performance, the proposed system constructs a multi-class classification architecture capable of mapping learners into four distinct outcome categories: Pass, Distinction, Withdrawn, and Fail. This granular categorization moves beyond binary pass/fail paradigms, offering a more nuanced understanding of how academic trajectories evolve throughout a course lifecycle.
- ❖ Beyond individual student forecasting, the comparative performance metrics derived from each trained model serve as diagnostic instruments for evaluating curriculum and instructional design effectiveness. Variations in model accuracy and feature importance scores across cohorts reveal which pedagogical elements, such as assessment frequency, content engagement depth, or interactive task completion, meaningfully influence learner achievement in competency-driven course structures.
- ❖ A foundational objective of this framework is the timely detection of academically vulnerable learners before withdrawal or failure becomes irreversible. By continuously validating model predictions against real-time student activity data, educators and academic advisors can be alerted to at-risk indicators sufficiently early in the academic term, enabling structured, evidence-based interventions tailored to individual learner needs rather than generalized remedial strategies.
- ❖ The proposed methodology is designed with scalability and transferability in mind, ensuring its applicability across diverse skill-based disciplines — including vocational training, technical certification programs, and professional upskilling courses. The model validation pipeline, grounded in historical data benchmarking, provides a replicable research architecture that institutions can adapt to their unique learner profiles, thereby contributing to the broader discourse on data-informed educational quality assurance.

ADVANTAGES OF PROPOSED SYSTEM:

- ❖ **Predictive Accuracy:** The system offers high predictive accuracy by leveraging various models, each excelling in forecasting different student outcome categories accurately.
- ❖ **Early Intervention:** By identifying at-risk students early, educators can intervene

timely, improving retention rates and overall student success in skill-based courses.

- ❖ **Personalized Learning Support:** The analysis enables the creation of personalized learning paths and support mechanisms for students, enhancing their engagement and learning outcomes.
- ❖ **Efficient Resource Allocation:** Institutions can efficiently allocate resources and support where needed most, optimizing the learning environment and support services for students.
- ❖ **Continuous Improvement:** The system facilitates the continuous improvement of instructional design through ongoing analysis of student performance and learning pathway efficacy.

3.3. HARDWARE & SOFTWARE REQUIREMENTS:

3.3.1. SOFTWARE REQUIREMENT SPECIFICATIONS:

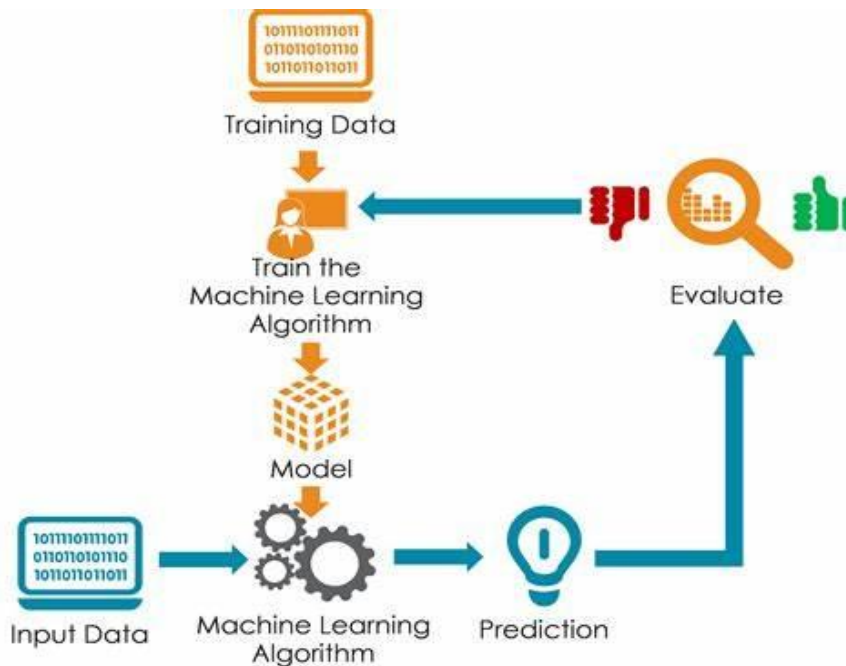
- | | |
|------------------------|-----------------------------|
| ❖ Operating System | : Windows 11 |
| ❖ Server side Script | : HTML, CSS, Bootstrap & JS |
| ❖ Programming Language | : Python |
| ❖ Libraries | : Django, |
| ❖ IDE/Workbench | : PyCharm |
| ❖ Technology | : Python 3.6+ |

3.3.2. HARDWARE REQUIREMENT SPECIFICATIONS:

- | | |
|-------------|-----------------------------|
| ❖ Processor | : I7/Intel Processor |
| ❖ Hard Disk | : 160GB |
| ❖ Key Board | : Standard Windows Keyboard |
| ❖ Mouse | : Two or Three Button Mouse |
| ❖ Monitor | : SVGA |
| ❖ RAM | : 8GB |

4. SYSTEM DESIGN

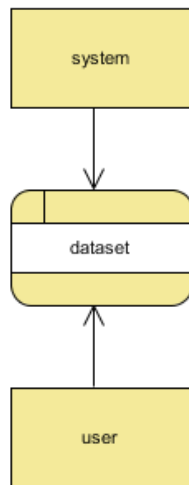
SYSTEM ARCHITECTURE:



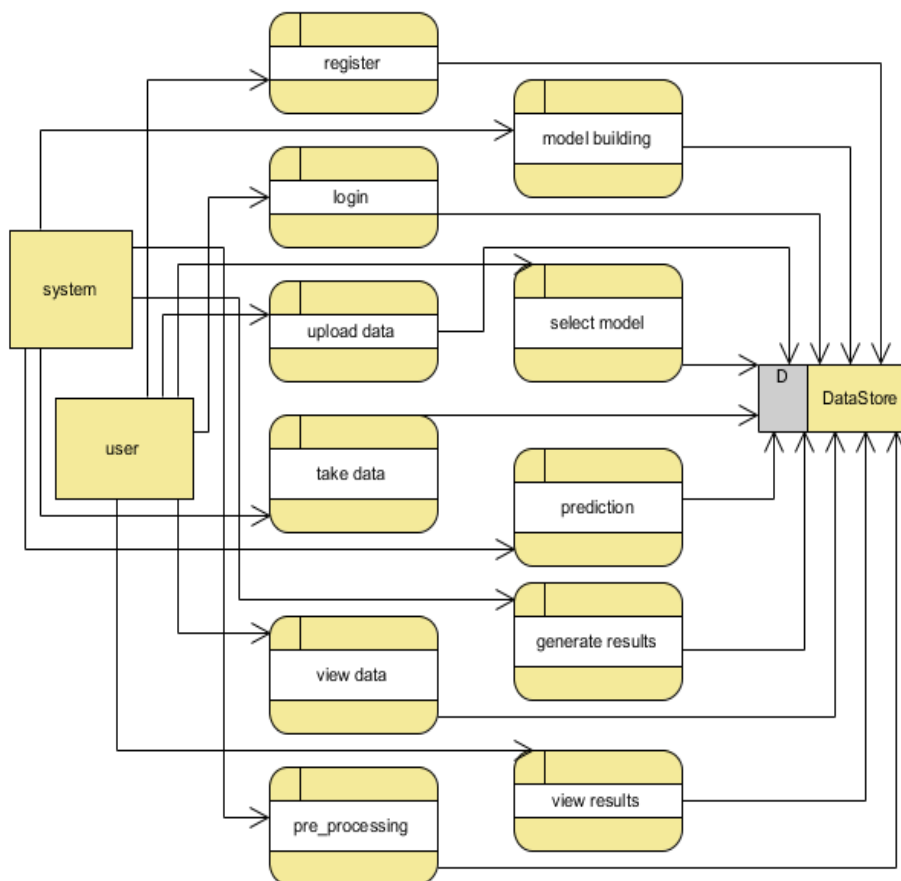
DATA FLOW DIAGRAM:

- ❖ The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
- ❖ The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
- ❖ DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
- ❖ DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.

Level 0:



Level 1:



UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

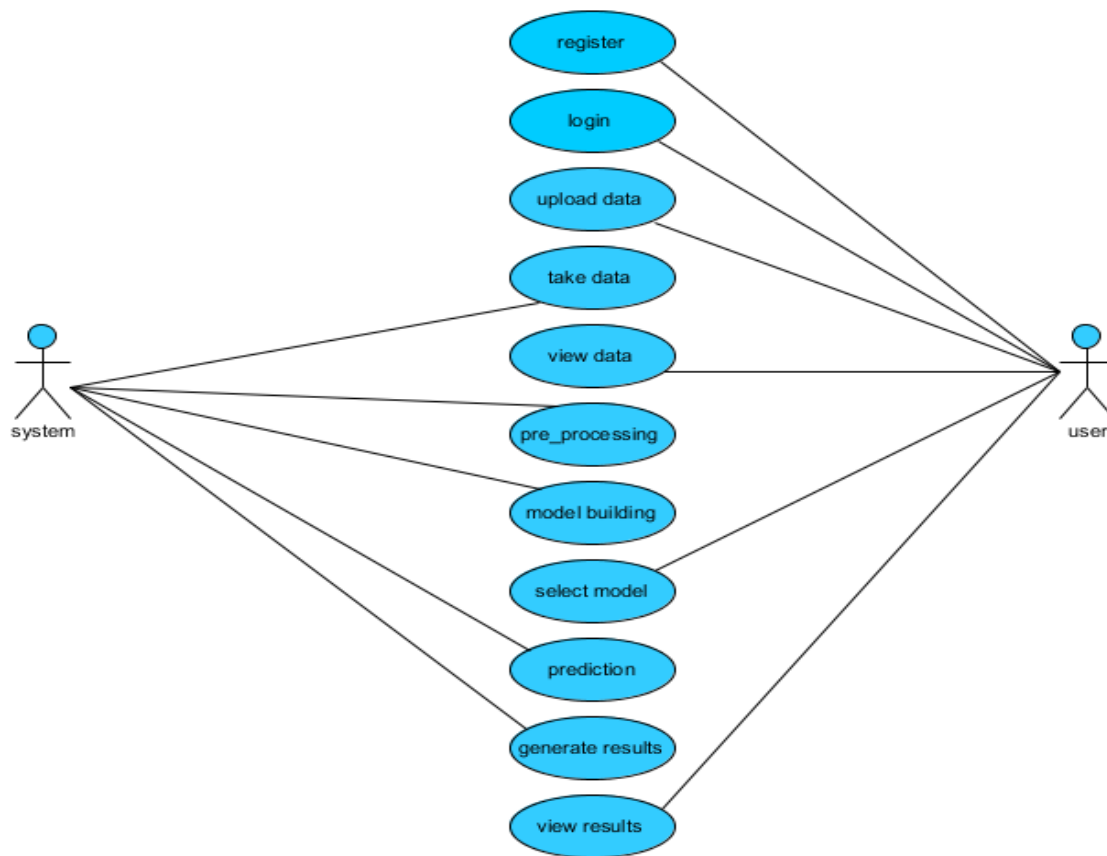
GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

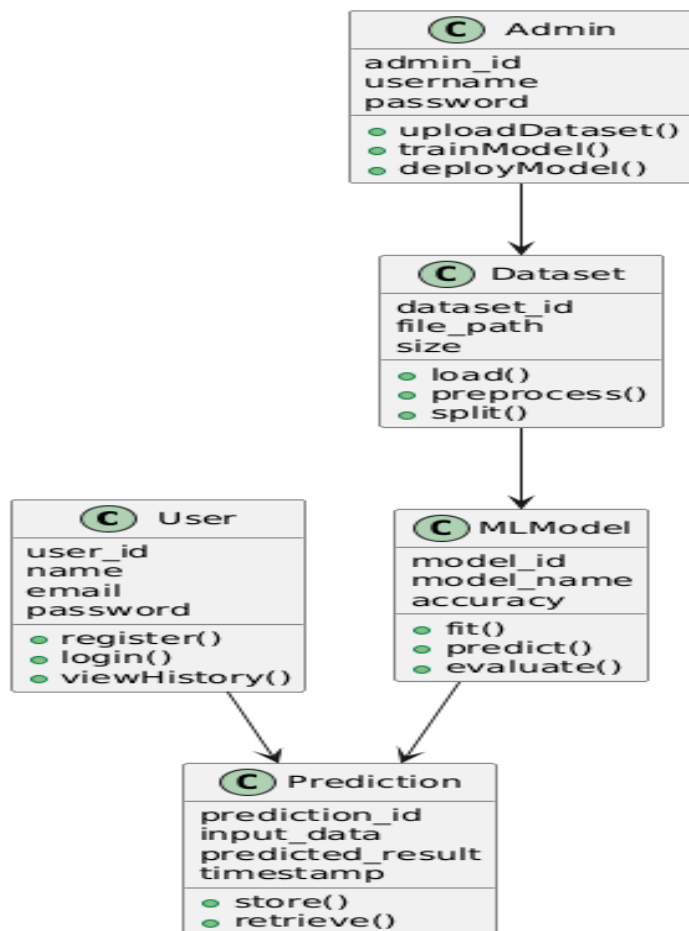
USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



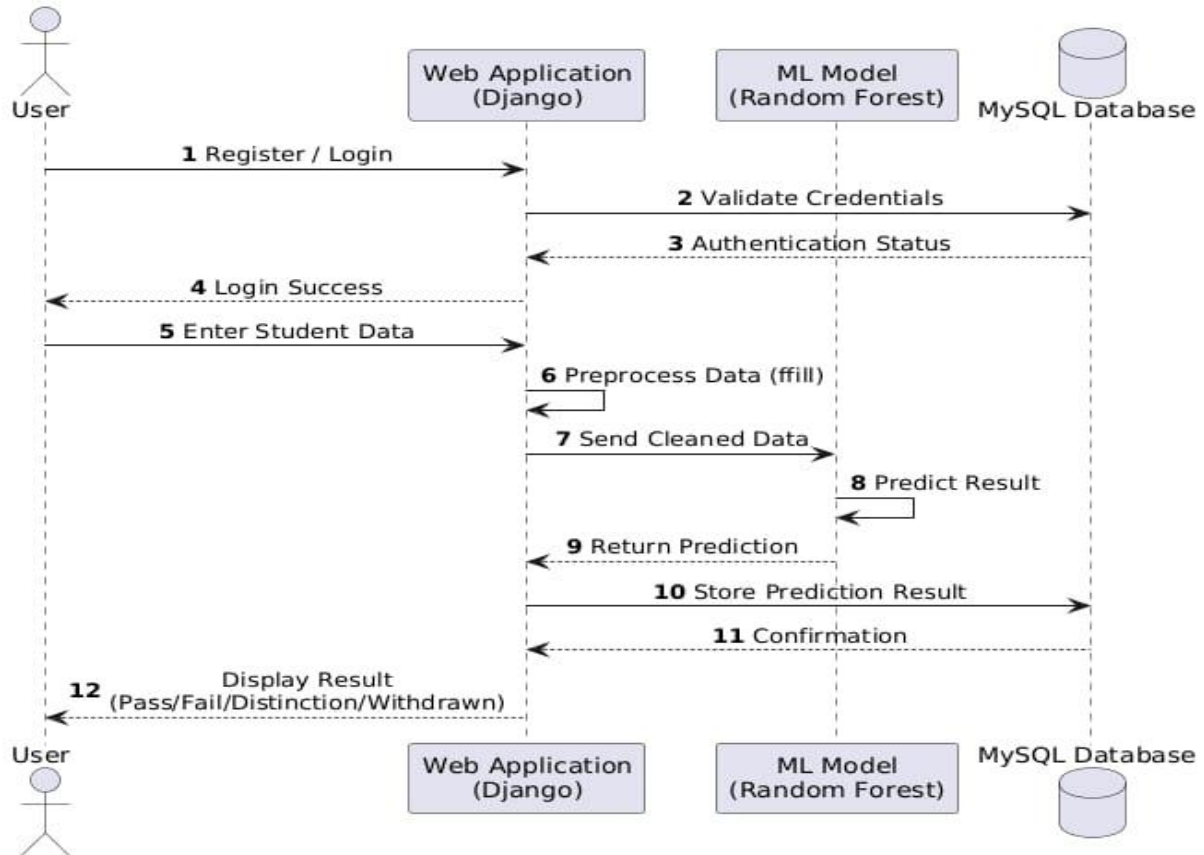
CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.



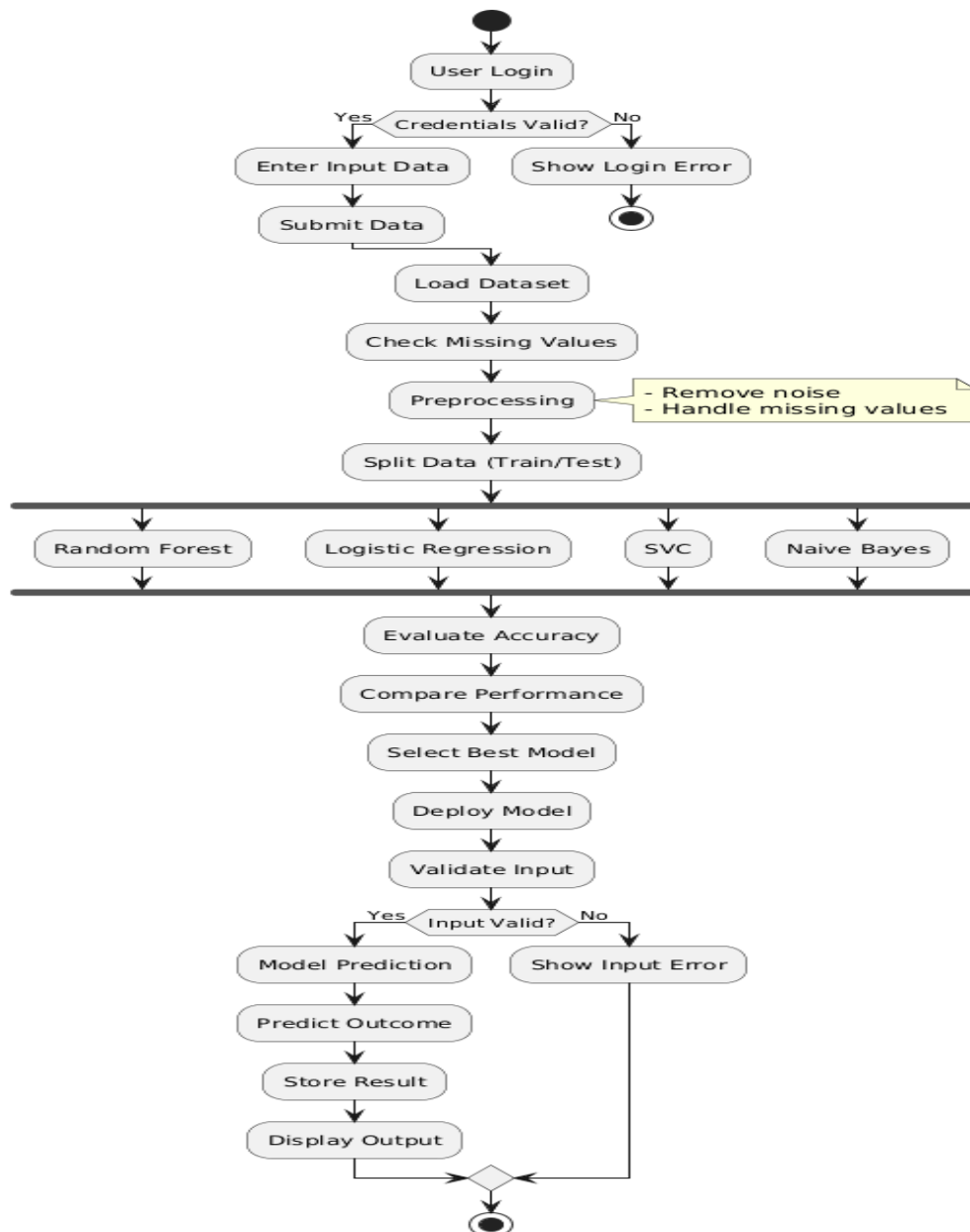
SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.



ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



5.IMPLEMENTATION

MODULES:

- ❖ User
 - Register
 - Login
 - View Data
 - Input
 - Result History
- ❖ Take Dataset
- ❖ Pre-Processing
- ❖ Training the data

MODULES DESCRIPTION:

User:

The User module is the main section of the Student Skill-Based Course Prediction web application where end-users directly interact with the system. It provides a simple and structured environment that guides users step by step from creating an account to receiving a predicted student outcome. This module is designed to be straightforward and easy to use, requiring no technical background or prior knowledge of machine learning. Whether the user is a student, educator, or administrator, this module ensures they can smoothly navigate through the application, enter the required information, and obtain meaningful predictions that support informed academic decision-making.

1.1 Registration:

The registration module serves as the foundational gateway through which prospective users gain authorized access to the Student Skill-Based Course Prediction web application. During this phase, users are prompted to submit essential identification credentials, thereby establishing a unique, secure profile within the system's database. This onboarding mechanism ensures that only authenticated individuals interact with sensitive academic datasets, maintaining data integrity and user accountability throughout the application's operational lifecycle.

1.2 Login:

Authentication and Personalized Portal Access Subsequent to successful registration, users are granted access to a personalized dashboard through a secure login interface. This authentication layer employs credential verification protocols to validate user identity before permitting entry into the prediction environment. Once logged in, the portal dynamically presents user-specific functionalities, ensuring that each individual operates within a tailored academic analytical space that reflects their interaction history and system privileges.

1.3 View Data:

Dynamic Input Parameter Configuration Users are facilitated with an interactive input interface through which they can manually submit relevant academic and behavioral parameters corresponding to a target student profile. These input values — encompassing variables such as engagement frequency, assessment performance indices, and course activity metrics — are channeled directly into the trained machine learning models. This human-in-the-loop input mechanism bridges the gap between raw data entry and intelligent algorithmic processing within the prediction framework.

1.4 Input Data:

Dynamic Input Parameter Configuration Users are facilitated with an interactive input interface through which they can manually submit relevant academic and behavioral parameters corresponding to a target student profile. These input values, encompassing variables such as engagement frequency, assessment performance indices, and course activity metrics — are channeled directly into the trained machine learning models. This human-in-the-loop input mechanism bridges the gap between raw data entry and intelligent algorithmic processing within the prediction framework.

1.5 Result History

Predictive Result Generation and Historical Outcome Tracking Upon submission of input parameters, the system deploys the pre-trained classification model to generate a predictive academic outcome, categorizing the student into one of four performance classes Pass, Fail, Withdrawn, or Distinction based on learned behavioral and assessment patterns. Furthermore, a dedicated result history module archives all prior prediction queries and their corresponding outputs, enabling users to longitudinally track performance forecasts, compare evolving

student profiles, and derive actionable insights for evidence-based academic intervention strategies.

Take Dataset:

The dataset for the OULA Data is collected from the kaggle website (kaggle.com).

The size of overall dataset is 20.7 MB (2,18,09,181 bytes)

Pre-Processing:

- In preprocessing first of all we will check whether there is any Nan values.
- If any Nan values is present we will fill the Nan values with different fillna techniques like bfill, ffill, mode, and mean.
- Here we used the ffill (front fill) technique on our project.

Training the data:

Irrespective of the algorithm we select the training is the same for every algorithm.

Given a dataset we split the data into two parts training and testing, the reason behind doing this is to test our model/algorithm performance just like the exams for a student the testing is also exam for the model.

We can split data into anything we want but it is just good practice to split the data such that the training has more data than the testing data, we generally split the data.

And for training and testing there are two variables X and Y in each of them, the X is the features that we use to predict the Y target and same for the testing also.

Then we call the .fit () method on any given algorithm which takes two parameters i.e., X and Y for calculating the math and after that when we call the .predict () giving our testing X as parameter and checking it with the accuracy score giving the testing Y and predicted X as the two parameters will get our accuracy score and same steps , these are just checking for how good our model performed on a given dataset.

SOFTWARE ENVIRONMENT

Python:

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted** – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** – You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented** – Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language** – Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

Python Features

Python's features include –

- **Easy-to-learn –Easy-to-read** – Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** – Python's source code is fairly easy-to-maintain.
- **A broad standard library** – Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.

- **Interactive Mode** – Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** – Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** – You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** – Python provides interfaces to all major commercial databases.
- **GUI Programming** – Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** – Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below –

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- It supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

Python is available on a wide variety of platforms including Linux and Mac OS X. Let's understand how to set up our Python environment.

Getting Python

The most up-to-date and current source code, binaries, documentation, news, etc., is available on the official website of Python <https://www.python.org>.

Windows Installation

Here are the steps to install Python on Windows machine.

- Open a Web browser and go to <https://www.python.org/downloads/>.
- Follow the link for the Windows installer python-XYZ.msifile where XYZ is the version you need to install.
- To use this installer python-XYZ.msi, the Windows system must support Microsoft Installer 2.0. Save the installer file to your local machine and then run it to find out if

your machine supports MSI.

- Run the downloaded file. This brings up the Python install wizard, which is really easy to use. Just accept the default settings, wait until the install is finished, and you are done.

The Python language has many similarities to Perl, C, and Java. However, there are some definite differences between the languages.

First Python Program

Let us execute programs in different modes of programming.

Interactive Mode Programming

Invoking the interpreter without passing a script file as a parameter brings up the following prompt –

```
$ python

Python2.4.3(#1,Nov112010,13:34:43)

[GCC 4.1.220080704(RedHat4.1.2-48)] on linux2

Type "help", "copyright", "credits" or "license" for more information.

>>>
```

Type the following text at the Python prompt and press the Enter –

```
>>>print"Hello, Python!"
```

If you are running new version of Python, then you would need to use print statement with parenthesis as in **print ("Hello, Python!");**. However in Python version 2.4.3, this produces the following result –

```
Hello, Python!
```

Script Mode Programming

Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.

Let us write a simple Python program in a script. Python files have extension **.py**. Type the following source code in a test.py file –

```
print"Hello, Python!"
```

We assume that you have Python interpreter set in PATH variable. Now, try to run this program as follows –

```
$ python test.py
```

This produces the following result –

```
Hello, Python!
```

Flask Framework:

Flask is a web application framework written in Python. Armin Ronacher, who leads an international group of Python enthusiasts named Pocco, develops it. Flask is based on Werkzeug WSGI toolkit and Jinja2 template engine. Both are Pocco projects.

Http protocol is the foundation of data communication in world wide web. Different methods of data retrieval from specified URL are defined in this protocol.

The following table summarizes different http methods

Sr.No	Methods & Description
1	GET Sends data in unencrypted form to the server. Most common method.
2	HEAD Same as GET, but without response body

3	POST Used to send HTML form data to server. Data received by POST method is not cached by server.
4	PUT Replaces all current representations of the target resource with the uploaded content.
5	DELETE Removes all current representations of the target resource given by a URL

By default, the Flask route responds to the **GET** requests. However, this preference can be altered by providing methods argument to **route()** decorator.

In order to demonstrate the use of **POST** method in URL routing, first let us create an HTML form and use the **POST** method to send form data to a URL.

Save the following script as login.html

```
<html>

<body>

<form action="http://localhost:5000/login" method="post">

<p>Enter Name:</p>

<p><input type="text" name="nm"/></p>

<p><input type="submit" value="submit"/></p>

</form>

</body>

</html>
```

Now enter the following script in Python shell.

```
from flask import Flask, redirect, url_for, request

app = Flask(__name__)

@app.route('/success/<name>')

def success(name):

    return 'welcome %s' % name

@app.route('/login', methods=['POST', 'GET'])

def login():

    if request.method == 'POST':

        user = request.form['nm']

        return redirect(url_for('success', name = user))

    else:

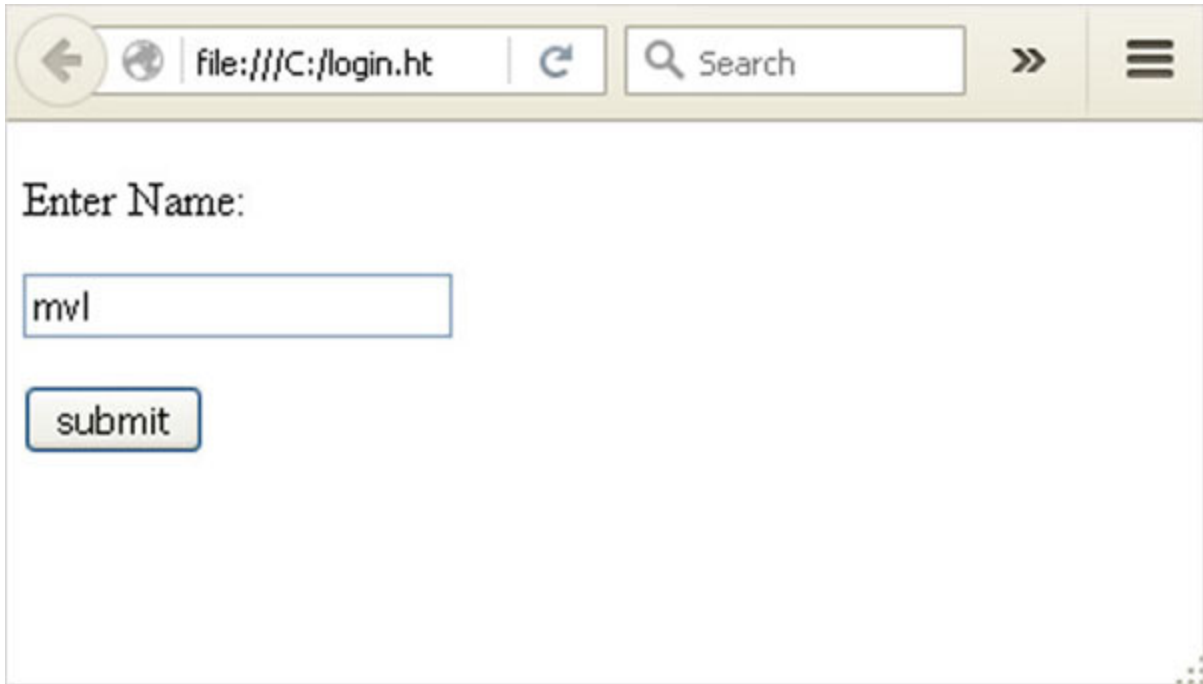
        user = request.args.get('nm')

        return redirect(url_for('success', name = user))

if __name__ == '__main__':

    app.run(debug = True)
```

After the development server starts running, open **login.html** in the browser, enter name in the text field and click **Submit**.



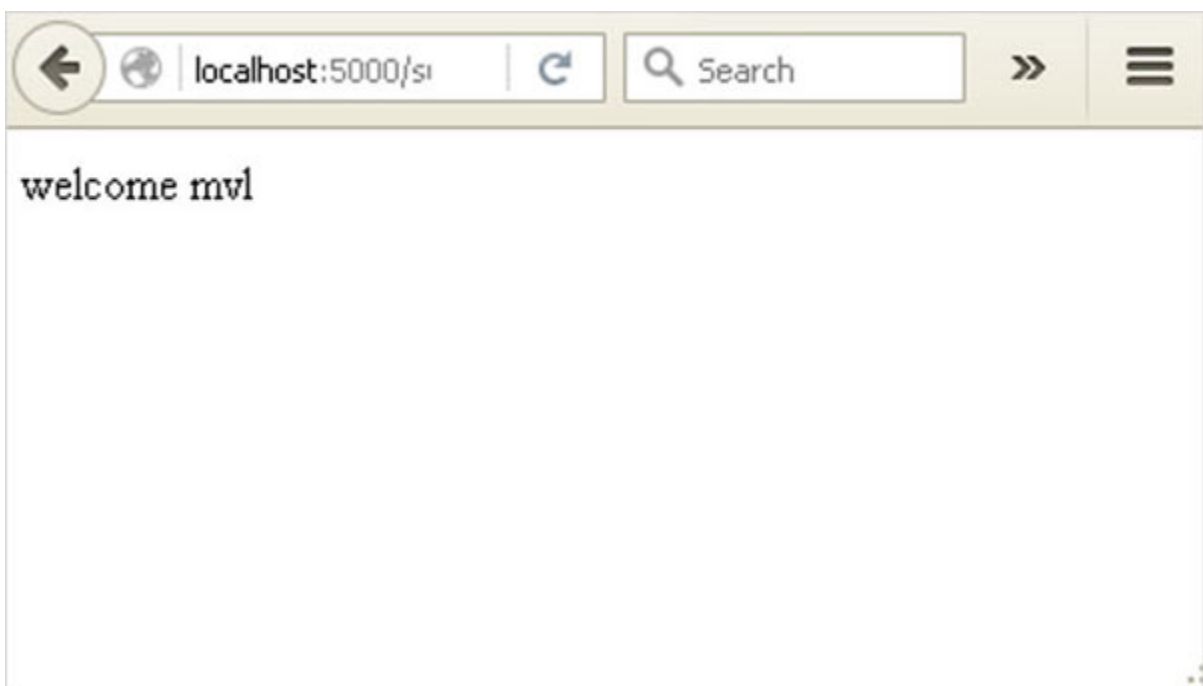
Enter Name:

Form data is POSTed to the URL in action clause of form tag.

http://localhost/login is mapped to the **login()** function. Since the server has received data by **POST** method, value of 'nm' parameter obtained from the form data is obtained by –

```
user = request.form['nm']
```

It is passed to '**success**' URL as variable part. The browser displays a **welcome** message in the window.



welcome mvl

Change the method parameter to '**GET**' in **login.html** and open it again in the browser. The

data received on server is by the **GET** method. The value of 'nm' parameter is now obtained by –

```
User = request.args.get('nm')
```

Here, **args** is dictionary object containing a list of pairs of form parameter and its corresponding value. The value corresponding to 'nm' parameter is passed on to '/success' URL as before.

What is Python?

Python is a popular programming language. It was created in 1991 by Guido van Rossum.

It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

What can Python do?

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.

- Python can be treated in a procedural way, an object-orientated way or a functional way.

Good to know

- The most recent major version of Python is Python 3, which we shall be using in this tutorial. However, Python 2, although not being updated with anything other than security updates, is still quite popular.
- In this tutorial Python will be written in a text editor. It is possible to write Python in an Integrated Development Environment, such as Thonny, Pycharm, Netbeans or Eclipse which are particularly useful when managing larger collections of Python files.

Python Syntax compared to other programming languages

- Python was designed to for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Python Install

Many PCs and Macs will have python already installed.

To check if you have python installed on a Windows PC, search in the start bar for Python or run the following on the Command Line (cmd.exe):

```
C:\Users\Your Name>python --version
```

To check if you have python installed on a Linux or Mac, then on linux open the command line or on Mac open the Terminal and type:

```
python --version
```

If you find that you do not have python installed on your computer, then you can download it for free from the following website: <https://www.python.org/>

Python Quick start

Python is an interpreted programming language, this means that as a developer you write Python (.py) files in a text editor and then put those files into the python interpreter to be executed.

The way to run a python file is like this on the command line:

```
C:\Users\Your Name>python helloworld.py
```

Where "helloworld.py" is the name of your python file.

Let's write our first Python file, called helloworld.py, which can be done in any text editor.

```
helloworld.py
```

```
print("Hello, World!")
```

Simple as that. Save your file. Open your command line, navigate to the directory where you saved your file, and run:

```
C:\Users\Your Name>python helloworld.py
```

The output should read:

```
Hello, World!
```

Congratulations, you have written and executed your first Python program.

The Python Command Line

To test a short amount of code in python sometimes it is quickest and easiest not to write the code in a file. This is made possible because Python can be run as a command line itself.

Type the following on the Windows, Mac or Linux command line:

```
C:\Users\Your Name>python
```

From there you can write any python, including our hello world example from earlier in the tutorial:

```
C:\Users\Your Name>python
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32
bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more
information.
>>> print("Hello, World!")
```

Which will write "Hello, World!" in the command line:

```
C:\Users\Your Name>python
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32
bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more
information.
>>> print("Hello, World!")
Hello, World!
```

Whenever you are done in the python command line, you can simply type the following to quit the python command line interface:

```
exit()
```

Execute Python Syntax

As we learned in the previous page, Python syntax can be executed by writing directly in the Command Line:

```
>>> print("Hello, World!")
```

Hello, World!

Or by creating a python file on the server, using the .py file extension, and running it in the Command Line:

```
C:\Users\Your Name>python myfile.py
```

Python Indentations

Where in other programming languages the indentation in code is for readability only, in Python the indentation is very important.

Python uses indentation to indicate a block of code.

Example

```
if 5 > 2:
```

```
    print("Five is greater than two!")
```

Python will give you an error if you skip the indentation:

Example

```
if 5 > 2:
```

```
print("Five is greater than two!")
```

Comments

Python has commenting capability for the purpose of in-code documentation.

Comments start with a #, and Python will render the rest of the line as a comment:

Example

Comments in Python:

```
#This is a comment.
```

```
print("Hello, World!")
```

Docstrings

Python also has extended documentation capability, called docstrings.

Docstrings can be one line, or multiline.

Python uses triple quotes at the beginning and end of the docstring:

6. ALGORITHMS

Algorithm:

1.SVC:

Support Vector Classification (SVC) is a pivotal technique in the realm of machine learning and data science, known for its effectiveness in classifying and predicting data categories. It primarily functions based on the Support Vector Machine (SVM) algorithm, which operates by finding the hyperplane that best segregates different classes in the feature space. One of the notable strengths of SVC is its proficiency in handling high-dimensional data, making it a favored choice for various classification tasks where the data set encompasses numerous features or variables.

SVC offers flexibility through the employment of different kernel functions, including linear, polynomial, radial basis function (RBF), and sigmoid. These kernel functions enable the algorithm to handle both linear and non-linear classification problems effectively, providing versatility in addressing diverse data sets and use cases. Furthermore, SVC is renowned for its robustness in avoiding overfitting, courtesy of the regularization parameters incorporated in its design. This characteristic ensures that the model generalizes well to unseen data, enhancing its reliability and accuracy in predictions.

Despite its advantages, SVC also comes with challenges like sensitivity to the choice of the kernel and hyperparameters, and it may require substantial computational resources, particularly for large datasets. Consequently, it is crucial for practitioners to carefully tune the model to optimize its performance for specific tasks, considering the inherent trade-offs in accuracy, computational efficiency, and complexity. Understanding these dynamics allows for the effective application of SVC in various domains, including the educational sector, where it can significantly contribute to learning analytics and student outcome predictions.

2.Random Forest :

A random forest is a machine learning technique that's used to solve regression and classification problems. It utilizes ensemble learning, which is a technique that combines many classifiers to provide solutions to complex problems.

A random forest algorithm consists of many decision trees. The 'forest' generated by the random forest algorithm is trained through bagging or bootstrap aggregating. Bagging is an

ensemble meta-algorithm that improves the accuracy of machine learning algorithms.

The (random forest) algorithm establishes the outcome based on the predictions of the decision trees. It predicts by taking the average or mean of the output from various trees.

Increasing the number of trees increases the precision of the outcome.

A random forest eradicates the limitations of a decision tree algorithm. It reduces the over fitting of datasets and increases precision. It generates predictions without requiring many configurations in packages (like Scikit-learn).

Features of a Random Forest Algorithm:

- It's more accurate than the decision tree algorithm.
- It provides an effective way of handling missing data.
- It can produce a reasonable prediction without hyper-parameter tuning.
- It solves the issue of over fitting in decision trees.
- In every random forest tree, a subset of features is selected randomly at the node's splitting point.

Decision trees are the building blocks of a random forest algorithm. A decision tree is a decision support technique that forms a tree-like structure. An overview of decision trees will help us understand how random forest algorithms work.

A decision tree consists of three components: decision nodes, leaf nodes, and a root node. A decision tree algorithm divides a training dataset into branches, which further segregate into other branches. This sequence continues until a leaf node is attained. The leaf node cannot be segregated further.

The nodes in the decision tree represent attributes that are used for predicting the outcome.

Decision nodes provide a link to the leaves. The following diagram shows the three types of nodes in a decision tree.

The information theory can provide more information on how decision trees work. Entropy and information gain are the building blocks of decision trees. An overview of these fundamental concepts will improve our understanding of how decision trees are built.

Entropy is a metric for calculating uncertainty. Information gain is a measure of how uncertainty in the target variable is reduced, given a set of independent variables.

The information gain concept involves using independent variables (features) to gain information about a target variable (class). The entropy of the target variable (Y) and the conditional entropy of Y (given X) are used to estimate the information gain. In this case, the

conditional entropy is subtracted from the entropy of Y .

Information gain is used in the training of decision trees. It helps in reducing uncertainty in these trees. A high information gain means that a high degree of uncertainty (information entropy) has been removed. Entropy and information gain are important in splitting branches, which is an important activity in the construction of decision trees.

Let's take a simple example of how a decision tree works. Suppose we want to predict if a customer will purchase a mobile phone or not. The features of the phone form the basis of his decision. This analysis can be presented in a decision tree diagram.

The root node and decision nodes of the decision represent the features of the phone mentioned above. The leaf node represents the final output, either buying or not buying. The main features that determine the choice include the price, internal storage, and Random Access Memory (RAM). The decision tree will appear as follows.

Applying decision trees in random forest

The main difference between the decision tree algorithm and the random forest algorithm is that establishing root nodes and segregating nodes is done randomly in the latter. The random forest employs the bagging method to generate the required prediction.

Bagging involves using different samples of data (training data) rather than just one sample.

A training dataset comprises observations and features that are used for making predictions.

The decision trees produce different outputs, depending on the training data fed to the random forest algorithm. These outputs will be ranked, and the highest will be selected as the final output.

Our first example can still be used to explain how random forests work. Instead of having a single decision tree, the random forest will have many decision trees. Let's assume we have only four decision trees. In this case, the training data comprising the phone's observations and features will be divided into four root nodes.

The root nodes could represent four features that could influence the customer's choice (price, internal storage, camera, and RAM). The random forest will split the nodes by selecting features randomly. The final prediction will be selected based on the outcome of the four trees.

The outcome chosen by most decision trees will be the final choice. If three trees predict buying, and one tree predicts not buying, then the final prediction will be buying. In this case, it's predicted that the customer will buy the phone.

3. logistic regression:

Logistic regression is a popular statistical method used in regression projects, particularly when dealing with binary or categorical outcomes. Unlike traditional linear regression, which aims to predict continuous numeric values, logistic regression is designed to model the probability of a binary event occurring. It is widely employed in various fields, such as healthcare, finance, marketing, and social sciences, where understanding and predicting binary outcomes are crucial.

The fundamental concept behind logistic regression lies in the use of the logistic function, also known as the sigmoid function, to transform the linear equation into a range between 0 and 1. This transformation allows us to interpret the output as a probability value. The model estimates the probability that an instance belongs to a specific class based on its input features.

During the model training process, the algorithm optimizes the coefficients (weights) for each feature to minimize the error between predicted probabilities and actual outcomes in the training data. This process is typically accomplished using optimization techniques like gradient descent.

Interpreting the coefficients in logistic regression is essential to understand the influence of each feature on the probability of the target class occurrence. Positive coefficients indicate that an increase in the corresponding feature raises the probability of the event happening, while negative coefficients suggest the opposite.

In regression projects, logistic regression can be used for tasks such as predicting customer churn, determining the likelihood of a disease diagnosis, fraud detection, and sentiment analysis. However, it's crucial to ensure data preprocessing, feature engineering, and model evaluation are appropriately carried out to achieve reliable and accurate results. Additionally, considering other advanced models like support vector machines or decision trees might be beneficial depending on the complexity of the problem at hand.

4. Naïve bayes:

Naïve Bayes is a probabilistic machine learning model used for classification tasks. It is based on applying Bayes' theorem, with an assumption of independence among predictors. The model is 'naïve' because it assumes that the presence of a feature in a class is unrelated to the presence of any other feature, even if these features are dependent on each other. Despite this simplification, Naïve Bayes classifiers often perform remarkably well in various

applications.

The model is particularly useful for large datasets due to its high speed and efficiency. It is often used for text classification purposes, including spam detection, sentiment analysis, and categorization of news articles. Naïve Bayes is also applied in the medical field for preliminary diagnosis based on various symptoms. Its simplicity makes it easy to interpret, and it requires relatively small amounts of data to produce accurate classifications.

Naïve Bayes classifiers are available in various types, including Gaussian Naïve Bayes, Multinomial Naïve Bayes, and Bernoulli Naïve Bayes. Each type is suited for different types of data distribution. Gaussian is used for normally distributed data, Multinomial is appropriate for discrete counts, and Bernoulli is useful for binary features.

Despite its advantages, one should be cautious of its limitations, like its assumption of feature independence and its sensitivity to irrelevant features. Understanding these limitations is crucial for effectively applying Naïve Bayes to real-world classification problems.

5. Cat Boost:

CatBoost is an efficient, open-source gradient boosting library, which is particularly adept at handling categorical features directly. Developed by Yandex, it is designed to efficiently handle large datasets, providing a robust and accurate method for classification and regression tasks. Its name is derived from “Category” and “Boosting,” highlighting its proficiency in working with categorical data without the necessity for extensive pre-processing.

This library stands out due to its superior performance in comparison with other popular gradient boosting libraries, like XGBoost and LightGBM. It utilizes a symmetric decision tree structure, which simplifies the process of splitting trees and subsequently results in faster and more accurate models. One of its significant features is the implementation of ordered boosting, a modification that helps combat overfitting, making it a reliable choice for various tasks.

Furthermore, CatBoost is user-friendly, offering various convenient tools for model interpretation and analysis. It supports visualization of training and is compatible with multiple programming languages including Python, R, and others. With its powerful capabilities and flexibility, CatBoost is a valuable asset for data scientists and researchers engaging in machine learning projects, especially those involving categorical data

C. CONCLUSION:

This study illuminates the efficacy of various machine learning models, including Random Forest, Logistic Regression, CatBoost, SVC, and Naïve Bayes, in analyzing student outcomes in skill-based courses. Each model demonstrated unique strengths in predicting outcome classes, providing crucial insights into student performance and instructional design effectiveness. The findings offer a valuable framework for identifying and assisting at-risk students early, promoting success in skill-based courses. This research significantly contributes to the fields of learning analytics and educational machine learning applications, showcasing their potential in creating supportive and adaptive learning environments for diverse student groups.

7. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub- assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the

Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

7.1 TYPES OF TESTS:

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as

specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

7.2 SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

8. RESULT

The use of machine learning models in exploring learning analytics of skill-based courses has demonstrated meaningful outcomes and holds strong potential for transforming student performance prediction and academic intervention strategies. Several key findings emerged from the comparative analysis of Random Forest, Logistic Regression, CatBoost, SVC, and Naïve Bayes:

Predictive Accuracy Across Models: Each machine learning model demonstrated distinct strengths in forecasting different student outcome categories Pass, Distinction, Withdrawn, and Fail. Rather than a one-size-fits-all solution, the comparative approach revealed that model selection significantly influences prediction reliability, with certain algorithms outperforming others depending on the nature of the student data and outcome class being predicted.

Early Identification of At-Risk Students: The framework proved effective in detecting students likely to underperform or withdraw before their situations become irreversible. By analyzing engagement levels and assessment scores from historical data, the system enables educators to intervene at the right time, improving overall retention rates and course completion outcomes in skill-based learning environments.

Personalized Learning and Support: The analytical outputs generated by the models go beyond mere classification they provide actionable insights that support the creation of individualized learning pathways. This allows institutions to tailor academic support mechanisms to each student's unique performance profile, enhancing both engagement and learning outcomes.

Instructional Design Evaluation: Beyond predicting student outcomes, the model results serve as diagnostic indicators of curriculum and instructional design effectiveness. Variations in model performance across student cohorts highlight which course components and pedagogical strategies most strongly influence learner success, helping educators refine their teaching approaches continuously.

Efficient Resource Allocation and Scalability: The proposed system equips educational institutions with a data-driven foundation for distributing academic resources more strategically directing support where it is genuinely needed. Furthermore, this research contributes to the broader field of learning analytics and machine learning in education, offering a replicable and scalable framework adaptable to diverse skill-based course environments and varied student populations.

9. CONCLUSION AND FUTURE WORK

In conclusion, the proposed system for analyzing student performance in skill-based courses using machine learning classification models has demonstrated significant effectiveness and reliability. By utilizing advanced algorithms such as Random Forest, Logistic Regression, Support Vector Classification (SVC), Naïve Bayes, and CatBoost, the system successfully predicts student outcomes into categories such as Pass, Distinction, Withdrawn, and Fail. The implementation of these models enables accurate and data-driven decision-making, helping educators identify at-risk students at an early stage. This early prediction capability allows institutions to provide timely support and intervention, ultimately improving student retention and academic success rates.

Additionally, the system offers objective evaluation by eliminating human bias and relying on data analytics for performance assessment. The comparative analysis of different machine learning models highlights their individual strengths and suitability for various prediction scenarios. Among them, ensemble models like Random Forest and boosting techniques such as CatBoost demonstrate strong performance in handling complex datasets.

Overall, this project showcases the potential of machine learning in transforming traditional educational systems into intelligent, adaptive, and student-centric learning environments. The integration of learning analytics with predictive modeling provides valuable insights that can enhance teaching strategies and improve overall educational outcomes.

Future Work:

Future enhancements for the project “Exploring the Learning Analytics of Skill-Based Course using Machine Learning Classification Models” can further improve its performance and applicability:

- ❖ **Incorporating Larger and Diverse Datasets:** Including data from multiple institutions and diverse student backgrounds can enhance the model’s accuracy and generalization capability.
- ❖ **Integration of Deep Learning Models:** Advanced techniques such as Artificial Neural Networks (ANN) and Deep Learning models can be incorporated to capture complex learning patterns more effectively.
- ❖ **Real-Time Performance Monitoring:** Developing a real-time monitoring system or dashboard can help track student progress continuously and provide instant feedback.

10. APPENDIX

10.1 APPENDIX:

In this appendix, a comparative analysis of various machine learning tools and technologies used in the project “Exploring the Learning Analytics of Skill-Based Course Using Machine Learning Classification Models” is presented. These tools play a crucial role in developing, training, and deploying machine learning models for analyzing student performance in skill-based courses.

1. Tool Name: Scikit-learn

- ❖ Type: Machine Learning Library (Python)
- ❖ Features: Classification, regression, clustering, preprocessing tools
- ❖ Integration: Easily integrates with NumPy, Pandas, and Matplotlib
- ❖ Advantages: Simple to use, efficient for small to medium datasets
- ❖ Scalability: Limited scalability for very large datasets
- ❖ Use in Project: Used for implementing models like Logistic Regression, SVC, and Naïve Bayes

2. Tool Name: Pandas

- ❖ Type: Data Analysis Library
- ❖ Features: Data cleaning, transformation, and analysis
- ❖ Integration: Works seamlessly with Python ML libraries
- ❖ Advantages: Easy handling of structured data
- ❖ Scalability: Moderate scalability
- ❖ Use in Project: Used for dataset preprocessing and handling missing values

3. Tool Name: NumPy

- ❖ Type: Numerical Computing Library
- ❖ Features: Mathematical operations, array manipulation
- ❖ Integration: Backbone for most ML libraries
- ❖ Advantages: High performance and efficiency
- ❖ Scalability: Suitable for large numerical computations
- ❖ Use in Project: Used for numerical data processing

4. Tool Name: Django

- ❖ Type: Web Framework
- ❖ Features: Backend development, authentication, database handling

- ❖ Integration: Integrates with ML models for deployment
- ❖ Advantages: Secure, scalable, and fast development
- ❖ Scalability: High scalability for web applications
- ❖ Use in Project: Used to build the web application for prediction system.

5. Tool Name: Matplotlib / Seaborn

- ❖ Type: Data Visualization Tools
- ❖ Features: Graphs, charts, data visualization
- ❖ Integration: Works with Pandas and NumPy
- ❖ Advantages: Helps in understanding data patterns
- ❖ Use in Project: Used for visualizing dataset and results

Organizing the Requirements

In this system, requirements are structured based on user roles and system functionality to ensure clarity and efficiency.

By User Type:

Admin: Responsible for managing datasets, training models, and monitoring system performance

User (Student): Provides input data and views prediction results

By System Mode

Certain systems operate differently depending on the mode in which they function. For instance, a system may perform distinct operations under different modes such as training mode, normal operation mode, or emergency mode. Each mode may have its own set of functionalities and behaviors.

By User Class

Some systems are designed to offer different functionalities based on user categories. For example, in a control system, different roles such as administrators, regular users, or maintenance staff may have access to different features and system capabilities depending on their responsibilities.

By Objects

Objects represent real-world entities that are modeled within the system. In a typical system, objects may include users, datasets, models, and processes. Each object contains specific attributes and performs certain functions. These functions, also known as services or methods, define how the object interacts within the system. Similar objects sharing common properties can be grouped into classes.

By Feature

A feature refers to a specific service or functionality provided by the system. Each feature may require a sequence of inputs and operations to produce the desired output. Features are typically defined using step-by-step input-output interactions, including validation checks, execution flow, handling of errors, and processing logic.

By Stimulus

In some cases, system requirements are better defined based on external inputs or events (stimuli). For example, a system may respond differently when receiving specific inputs such as user data submission, system errors, or unexpected conditions. Each stimulus triggers a corresponding system function.

By Response

Another way to organize requirements is by focusing on the outputs or responses generated by the system. In this approach, system functionalities are grouped based on the type of output they produce, such as generating predictions, displaying results, or producing reports.

By Functional Hierarchy

If the above approaches are not sufficient, the system can be structured using a functional hierarchy. In this method, functions are organized based on shared inputs, outputs, or internal data dependencies. Tools such as data flow diagrams and data dictionaries can be used to represent the relationships between different system components.

Additional Comments

When preparing a requirements specification, it is often beneficial to combine multiple organizational approaches. Depending on the complexity of the system, different structures can be used simultaneously to improve clarity and understanding.

Various methods, notations, and tools are available to support requirement documentation. Their effectiveness largely depends on how well the requirements are structured. For instance, state diagrams can be used when organizing by system mode, object-oriented techniques for object-based organization, sequence-based descriptions for features, and data flow diagrams for functional hierarchies.

CASE

Computer-Aided Software Engineering (CASE) refers to the use of software tools that assist in the development, design, and maintenance of software systems. These tools are commonly known as CASE tools and are used to support different activities throughout the software development lifecycle.

CASE Tool

- ❖ A CASE tool is a computer-based application designed to support one or more activities involved in software engineering during the development process.
- ❖ CASE tools are used across all phases of information system development, including analysis, design, and programming. For instance, diagramming tools and data dictionaries support the analysis and design phases, while code generators help speed up the programming phase.
- ❖ These tools provide automated support for designing and documenting structured programming approaches. The primary goal of CASE is to create a system description that is detailed enough to generate the required software components automatically.

CLASSIFICATION of CASE TOOLS

Existing CASE tools can be classified along 4 different dimensions:

1. Life-cycle support
2. Integration dimension
3. Construction dimension
4. Knowledge-based CASE dimension

Let us take the meaning of these dimensions along with their examples one by one:

Life-Cycle Based CASE Tools

This classification is based on the stage of the software development life cycle that the tool supports. These tools are generally divided into Upper CASE and Lower CASE tools.

UpperCASE Tool

UpperCASE Tool is a Computer-Aided Software Engineering (CASE) software tool that supports the software development activities upstream from implementation.

UpperCASE tool focus on the analysis phase (but sometimes also the design phase) of the software development lifecycle (diagramming tools, report and form generators, and analysis tools)

LowerCASE Tool

LowerCASE Tool Computer-Aided Software Engineering (CASE) software tool that directly supports the implementation (programming) and integration tasks.

LowerCASE tools support database schema generation, program generation, implementation, testing, and configuration management.

Integration dimension

Three main CASE Integration dimensions have been proposed:

1. CASE Framework

2. ICASE Tools Tools that integrate both upper and lower CASE, for example making it possible to design a form and build the database to support it at the same time. An automated system development environment that provides numerous tools to create diagrams, forms and reports. It also offers analysis, reporting, and code generation facilities and seamlessly shares and integrates data across and between tools.

3. Integrated Project Support Environment(IPSE)

Types of CASE Tools The general types of CASE tools are listed below:

1. Diagramming tools: enable system process, data and control structures to be represented graphically.
2. Computer display and report generators: help prototype how systems look and feel. It makes it easier for the systems analyst to identify data requirements and relationship.
3. Analysis tools: automatically check for importance, inconsistent, or incorrect specifications in diagrams, forms, and reports.
4. Central repository: enables the integrated storage of specifications, diagrams, reports and project management information.
5. Documentation Generators: produce technical and user documentation in standard formats.
6. Code generators: enable the automatic generation of program and data base definition code directly from the design documents, diagrams, forms, and reports.

Functions of a CASE Tool

1. Analysis CASE analysis tools automatically check for incomplete, inconsistent, or in correct specifications in diagrams, forms and reports.
2. Design This is where the technical blueprint of the system is created by designing the technical architecture – choosing amongst the architectural designs of telecommunications, hardware and software that will best suit the organization's system and future needs. Also designing the systems model – graphically creating a model from graphical user interface, screen design, and databases, to placement of objects on screen
3. Code generation CASE Tool has code generators which enable the automatic generation of program and data base definition code directly from the documents, diagrams, forms, and reports.
4. Documentation CASE Tool has documentation generators to produce technical and user documentation in standard forms. Each phase of the SDLC produces documentation. The types of documentation that flow from one face to the next vary depending upon the organization, methodologies employed and type of system being

built.

CASE Environments

An environment is a collection of CASE tools and workbenches that supports the software process. CASE environments are classified based on the focus/basis of integration

1. Toolkits
2. Language-centered
3. Integrated
4. Fourth generation
5. Process-centered

Toolkits

Toolkits are loosely integrated collections of products easily extended by aggregating different tools and workbenches. Typically, the support provided by a toolkit is limited to programming, configuration management and project management. And the toolkit itself is environments extended from basic sets of operating system tools, for example, the Unix Programmer's Work Bench and the VMS VAX Set. In addition, toolkits' loose integration requires user to activate tools by explicit invocation or simple control mechanisms. The resulting files are unstructured and could be in different format, therefore the access of file from different tools may require explicit file format conversion. However, since the only constraint for adding a new component is the formats of the files, toolkits can be easily and incrementally extended.

Language-centered

The environment itself is written in the programming language for which it was developed, thus enabling users to reuse, customize and extend the environment. Integration of code in different languages is a major issue for language-centered environments. Lack of process and data integration is also a problem. The strengths of these environments include good level of presentation and control integration. Interlisp, Smalltalk, Rational, and KEE are examples of language-centered environments.

Integrated

These environments achieve presentation integration by providing uniform, consistent, and coherent tool and workbench interfaces. Data integration is achieved through the repository concept: they have a specialized database managing all information produced and accessed in the environment. Examples of integrated environment are the ICL CADE system, IBM AD/Cycle and DEC Cohesion.

Fourth-generation

Fourth-generation environments were the first integrated environments. They are sets of tools and workbenches supporting the development of a specific class of program: electronic data processing and business-oriented applications. In general, they include programming tools, simple configuration management tools, document handling facilities and, sometimes, a code generator to produce code in lower-level languages. Informix 4GL, and Focus fall into this category.

Process-centered

Environments in this category focus on process integration with other integration dimensions as starting points. A process-centered environment operates by interpreting a process model created by specialized tools. They usually consist of tools handling two functions:

- Process-model execution
- Process-model production

Examples are East, Enterprise II, Process Wise, Process Weaver, and Arcadia

CASE Tools ADVANTAGES

DISADVANTAGES

Helps standardization of notations and diagrams

Limitations in the flexibility of documentation

Help communication between development team members

May lead to restriction to the tool's capabilities

Automatically check the quality of the models

Major danger: completeness and syntactic correctness does NOT mean compliance with requirements

Reduction of time and effort

Costs associated with the use of the tool:
purchase + training

Enhance reuse of models or models

10.2 SAMPLECODE:

```
from django.shortcuts import render, redirect
from django.contrib.auth.models import User
import pandas as pd
from .mtrain import ModuleTrain
import pickle

# Create your views here.

class Home:

    def index(req):

        return render(req, 'index.html')

    def about(req):

        return render(req, 'index.html', {'msg': 'about'})

class RegLog:

    def login(req):

        if req.method == 'POST':

            uemail = req.POST['uemail']

            upass = req.POST['upass']

            user = User.objects.filter(email=uemail, password=upass).exists()

            if user:

                return render(req, 'userhome.html')

            else:

                return render(req, 'index.html', {'msg': 'login'})

        return render(req, 'index.html', {'msg': 'login'})

    def register(req):

        if req.method == 'POST':
```

```
uname = req.POST['uname']
uemail = req.POST['uemail']
upass = req.POST['upass']
cpass = req.POST['cpass']
if upass == cpass:
    User.objects.create(username=uname, email=uemail, password=upass)
    return render(req, 'index.html', {'msg': 'login', 'msg1': 'registersuccess'})
return render(req, 'index.html', {'msg': 'register'})

def check_email(req):
    if req.method == 'POST':
        uemail = req.POST['uemail']
        ema = User.objects.filter(email=uemail).exists()
        if ema:
            em = req.session['email'] = uemail
            print(em)
            return render(req, 'index.html', {'msg': 'changepass'})
        return render(req, 'index.html', {'msg': 'check_email'})

def forgot(req):
    if req.method == 'POST':
        upass = req.POST['upass']
        cpass = req.POST['cpass']
        if upass == cpass:
            pa = User.objects.get(email=req.session['email'])
            pa.password = upass
            pa.save()
```

```
        return render(req, 'index.html', {'msg': 'login', 'msg1': 'passwordchange'})

    return render(req, 'index.html', {'msg': 'change pass'})

class UserHome:

    def view(req):

        df = pd.read_csv('DATASET/final_merged_dataset.csv')

        data = df.head(100).to_html()

        return render(req, 'userhome.html', {'msg': 'view', 'data': data})

    def train(req):

        if req.method == 'POST':

            algo = req.POST['algo']

            if algo == '1':

                report = ModuleTrain.svca()

                return render(req, 'userhome.html', {'msg': 'train', 'report': report, 'algo': 'SVC'})

            elif algo == '2':

                report = ModuleTrain.rfca()

                return render(req, 'userhome.html', {'msg': 'train', 'report': report, 'algo': 'Random
Forest'})

            elif algo == '3':

                report = ModuleTrain.nbca()

                return render(req, 'userhome.html', {'msg': 'train', 'report': report, 'algo': 'Naive
Bayes'})

            elif algo == '4':

                report = ModuleTrain.lrca()

                return render(req, 'userhome.html', {'msg': 'train', 'report': report, 'algo': 'Logistic
Regression'})

            elif algo == '5':

                report = ModuleTrain.caca()

                return render(req, 'userhome.html', {'msg': 'train', 'report': report, 'algo': 'CatBoost'})
```

```
    return render(req, 'userhome.html', {'msg': 'train'})

def pred(req):
    if req.method == 'POST':
        global inp
        dic = req.POST.dict()
        del dic['csrfmiddlewaretoken']

        # FIX: check for empty fields before converting
        for key, value in dic.items():
            if value == "" or value is None:
                return render(req, 'pred.html', {
                    'error': f'Please fill in the field: {key}'
                })

        inp = []
        for i in dic.keys():
            inp.append(float(dic[i]))

        return redirect(UserHome.output)

    return render(req, 'pred.html')

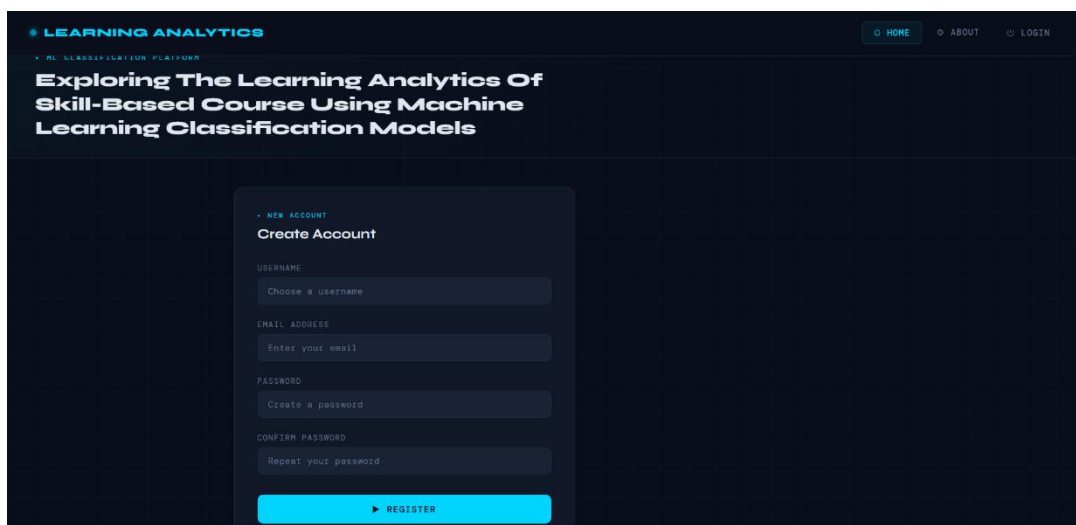
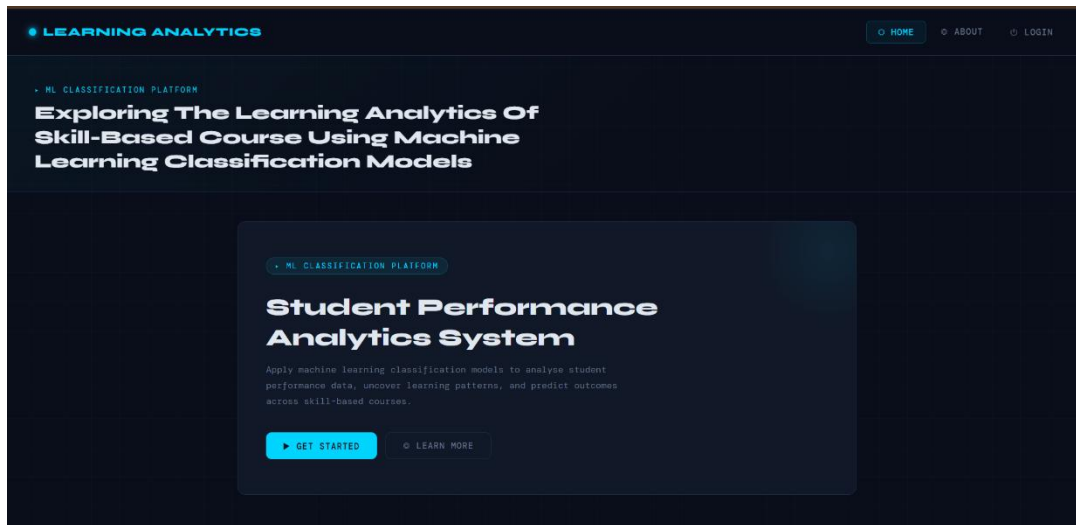
def output(req):
    algo_index = inp[18]
    features = [inp[:18]]

    if algo_index == 0:
        with open('ALGORITHM/SVC.algo', 'rb') as f:
            mod = pickle.load(f)
```

```
elif algo_index == 1:
    with open('ALGORITHM/RF.algo', 'rb') as f:
        mod = pickle.load(f)
elif algo_index == 2:
    with open('ALGORITHM/nb.algo', 'rb') as f:
        mod = pickle.load(f)
elif algo_index == 3:
    with open('ALGORITHM/lr.algo', 'rb') as f:
        mod = pickle.load(f)
elif algo_index == 4:
    with open('ALGORITHM/ca.algo', 'rb') as f:
        mod = pickle.load(f)

result = mod.predict(features)
print(result)
return render(req, 'pred.html', {'msg': int(result[0])})
```

10.3 SCREEN SHOTS OF MODULES



• LEARNING ANALYTICS

HOME ABOUT LOGIN

• ML CLASSIFICATION PLATFORM

Exploring The Learning Analytics Of Skill-Based Course Using Machine Learning Classification Models

• SECURE ACCESS

Sign In

EMAIL ADDRESS

Enter your email

PASSWORD

Enter your password

☐ Keep me signed in

SIGN IN

[Forgot password?](#) [Create account](#)

• LEARNING ANALYTICS

HOME DATASET TRAIN PREDICT LOGOUT

• ML CLASSIFICATION DASHBOARD

Exploring The Learning Analytics Of Skill-Based Course Using Machine Learning Classification Models

TOTAL RECORDS

32K+

Merged dataset

FEATURES

18

Input variables

ALGORITHMS

3

Classifiers available

TARGET CLASSES

4

Pass / Fail / Distinction / Withdrawn

View Dataset

Browse All records

Train Models

Run classifiers

Predict

Classify new input

• LEARNING ANALYTICS

HOME DATASET TRAIN PREDICT LOGOUT

Skill-Based Course Using Machine Learning Classification Models

Dataset Preview

Showing first 100 records from final_merged_dataset.csv

100 ROWS

Filter rows...

Scroll horizontally to see all columns

	CODE_MODULE	CODE_PRESENTATION	ID_ASSESSMENT	ASSESSMENT_TYPE	DATE	WEIGHT	ID_STUDENT	DATE_SUBMITTED	IS_BANNED	SCORE	MODULE_PRESENTATION_LENGTH	GENDER	REGI
0	AAA	2013J	1752	TMA	19.0	10.0	11391	18	0	70.0	268	M	East
1	AAA	2013J	1753	TMA	54.0	20.0	11391	53	0	85.0	268	M	East
2	AAA	2013J	1754	TMA	117.0	20.0	11391	115	0	80.0	268	M	East
3	AAA	2013J	1750	TMA	186.0	20.0	11391	184	0	85.0	268	M	East
4	AAA	2013J	1756	TMA	215.0	30.0	11391	212	0	82.0	268	M	East
5	AAA	2013J	1752	TMA	19.0	10.0	28400	22	0	70.0	268	F	Scot
6	AAA	2013J	1753	TMA	54.0	20.0	28400	52	0	68.0	268	F	Scot
7	AAA	2013J	1754	TMA	117.0	20.0	28400	121	0	70.0	268	F	Scot
8	AAA	2013J	1755	TMA	166.0	20.0	28400	164	0	64.0	268	F	Scot
9	AAA	2013J	1756	TMA	215.0	30.0	28400	212	0	68.0	268	F	Scot
10	AAA	2013J	1752	TMA	19.0	10.0	31604	17	0	72.0	268	F	Scot

LEARNING ANALYTICSHOMEDATASETTRAINPREDICTLOGOUT

AAA

CODE PRESENTATION
2013b

ID ASSESSMENT
e.g. 1752

ASSESSMENT TYPE
CHA

WEIGHT
e.g. 10.0

ID STUDENT
e.g. 28400

e.g. 18

IS BANKED
0 or 1

SCORE
e.g. 78.0

MODULE PRESENTATION LENGTH
e.g. 269

GENDER
Female

REGION
East Anglian Region

A Level or Equivalent

AGE BAND
0 to 35

NUM OF PREV ATTEMPTS
0 or 1

STUDIED CREDITS
e.g. 120

DISABILITY
0 or 1

NUM ACTIVITY TYPES
e.g. 6

Select Algorithm

SVC

Random

Naive

Logistic

CatBoost

READY TO CLASSIFY

Fill in all fields across the three sections, then run the selected algorithm to predict the student's outcome.

RUN PREDICTION

LEARNING ANALYTICSHOMEDATASETTRAINPREDICTLOGOUT

ID ASSESSMENT
e.g. 1752

ASSESSMENT TYPE
CHA

WEIGHT
e.g. 10.0

ID STUDENT
e.g. 28400

SCORE
e.g. 78.0

MODULE PRESENTATION LENGTH
e.g. 269

GENDER
Female

REGION
East Anglian Region

NUM OF PREV ATTEMPTS
0 or 1

STUDIED CREDITS
e.g. 120

DISABILITY
0 or 1

NUM ACTIVITY TYPES
e.g. 6

Select Algorithm

SVC
Kernel-based

Random Forest
Ensemble

Naive Bayes
Probabilistic

Logistic Reg.
Linear model

CatBoost
Gradient boost

READY TO CLASSIFY

Fill in all fields across the three sections, then run the selected algorithm to predict the student's outcome.

RUN PREDICTION

PREDICTION RESULT

Withdrawn

11. REFERENCES

- [1] Akoglu H. User's guide to correlation coefficients. Turkish journal of emergency medicine. 2018 Sep 1;18(3):91-3.
- [2] Aljohani, N. R., & Davis, H. C. (2012, September). Significance of learning analytics in enhancing the mobile and pervasive learning environments. In 2012 Sixth International Conference on Next Generation Mobile Applications, Services and Technologies (pp. 70-74). IEEE.
- [3] Chickering, Arthur W. "Experience and Learning. An Introduction to Experiential Learning." (1977)
- [4] Gentry, James W. "What is experiential learning." Guide to business gaming and experiential learning 9 (1990): 20.
- [5] Heuer, H., & Breiter, A. (2018). Student success prediction and the trade off between big data and data minimization. DeLFI 2018-Die 16. ELearning Fachtagung Informatik.
- [6] Jantjies, Mmaki, Trevor Moodley, and Ronel Maart. "Experiential learning through virtual and augmented reality in higher education." Proceedings of the 2018 international conference on education technology management. 201
- [7] Maulud, D., & Abdulazeez, A. M. (2020). A review on linear regression comprehensive in machine learning. Journal of Applied Science and Technology Trends, 1(4), 140-147.
- [8] Peach, R. L., Yaliraki, S. N., Lefevre, D., & Barahona, M. (2019). Datadriven unsupervised clustering of online learner behaviour. npj Science of Learning, 4(1), 14.
- [9] Schober, P., Boer, C., & Schwarte, L. A. (2018). Correlation coefficients: appropriate use and interpretation. Anesthesia & analgesia, 126(5), 1763-1768.
- [10] Waheed, H., Hassan, S. U., Aljohani, N. R., Hardman, J., Alelyani, S., & Nawaz, R. (2020). Predicting academic performance of students from VLE big data using deep learning

models. *Computers in Human behavior*, 104, 106189

[11] Wasif, M., Waheed, H., Aljohani, N. R., & Hassan, S. U. (2019). Understanding student learning behavior and predicting their performance. In *Cognitive Computing in Technology-Enhanced Learning* (pp. 1-28). IGI Global.

[12] Wu, J., Liu, C., Cui, W., & Zhang, Y. (2019, November). Personalized Collaborative Filtering Recommendation Algorithm based on Linear Regression. In *2019 IEEE International Conference on Power Data Science (ICPDS)* (pp. 139-142). IEEE.